

Code Complexity Analysis US007

Index

1	Pseudocode	2
1.1	Function 1: TwoDTree	2
1.2	Function 2: insertHelper	2
1.3	Function 3: inOrderTransversalData	2
1.4	Function 4: inOrderTransversalHelperData	2
2	Time Complexity	3
2.1	Function: TwoDTree	3
2.2	Function: insertHelper	3
2.3	Function: inOrderTransversalData	3
2.4	Function: inOrderTransversalHelperData	3

1 Pseudocode

1.1 Function 1: TwoDTree

Location: TwoDTree Implementation

```
1 FUNCTION TwoDTree(AVL tree):  
2     List of node datas From AVL = avl.inOrderTransversalData()  
3     Sort by first comparator(x)  
4  
5     root = insertHelper(node list, depth)  
6     END FUNCTION
```

1.2 Function 2: insertHelper

Location: TwoDTree Implementation

```
1 FUNCTION insertHelper(NodeData list, depth):  
2     IF list is NULL || List is over  
3         END  
4  
5     Increment size  
6     Sort List according to depth(x or y)  
7  
8     Get midian index and create this node  
9  
10    split the list in half(using median index)  
11  
12    node.setLeft (insertHelper(leftSubList, depth + 1))  
13    node.setRight (insertHelper(rightSubList, depth + 1))  
14    END FUNCTION
```

1.3 Function 3: inOrderTransversalData

Location: AVL Tree Implementation

```
1 FUNCTION inOrderTransversalData():  
2     Start list of nodes  
3  
4     call helper(inOrderTransversalHelperData(list, tree_root))  
5     END FUNCTION
```

1.4 Function 4: inOrderTransversalHelperData

Location: AVL Tree Implementation

```
1 Functions inOrderTransversalHelperData(List nodes, Node node):  
2     IF node is NULL  
3         END  
4  
5     inOrderTransversalHelperData(List, left node)  
6     nodes.add(node element)  
7     inOrderTransversalHelperData(List, Right node)  
8     END FUNCTION
```

2 Time Complexity

2.1 Function: TwoDTree

Operation	Description	Best Case	Average Case	Worst Case	Notes
inOrderTransversal	Extract all nodes from AVL	$O(n)$	$O(n)$	$O(n)$	Linear traversal
insertHelper	Build 2D tree	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	One visit per element
TOTAL		$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	Sorting dominates

2.2 Function: insertHelper

Operation	Description	Best Case	Average Case	Worst Case	Notes
Base case	Empty list check	$O(1)$	$O(1)$	$O(1)$	Stops recursion
Sorting list	Sort by selected axis	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	Happens at each recursive level
Build node	Create median node	$O(1)$	$O(1)$	$O(1)$	Constant
Split list	Partition into halves	$O(n)$	$O(n)$	$O(n)$	Dependent on list size
Recursive calls	Two subtree constructions	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	Height $\times$ sort cost
TOTAL		$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	

2.3 Function: inOrderTransversalData

Operation	Description	Best Case	Average Case	Worst Case	Notes
Init list	Create empty list	$O(1)$	$O(1)$	$O(1)$	Constant
Call helper	Traverse entire AVL	$O(n)$	$O(n)$	$O(n)$	Visits all nodes
Return list	Return result	$O(1)$	$O(1)$	$O(1)$	Constant
TOTAL		$O(n)$	$O(n)$	$O(n)$	Linear time

2.4 Function: inOrderTransversalHelperData

Operation	Description	Best Case	Average Case	Worst Case	Notes
Null check	Base case	$O(1)$	$O(1)$	$O(1)$	Constant

Left recursion	Visit left subtree	$O(n)$	$O(n)$	$O(n)$	Part of full traversal
Add element	Insert node data	$O(1)$	$O(1)$	$O(1)$	Constant
Right recursion	Visit right subtree	$O(n)$	$O(n)$	$O(n)$	Part of full traversal
TOTAL		$O(n)$	$O(n)$	$O(n)$	Every node visited once